

ARTH-AI

PROJECT COMPLETION PLAN

What is complete, what remains, and the exact path to a submission-ready build.

Date: 12 September 2026
Repository: https://github.com/snehpate105/HackOut-26.git
Current branch: main
Current checkpoint: c9bd102 Add Alembic database migrations

Executive Summary

Arth-AI is a working hackathon vertical slice for hyper-personalized banking in Bharat. The repository contains a FastAPI backend, Next.js PWA, PostgreSQL/pgvector Docker stack, seeded personas, fraud scoring, life-stage classification, ethics rules, chat fallback, mock KYC, explainability, and a judge-facing transaction simulator.

The project is not yet submission-complete because the full stack has not been run on a machine with Node.js and Docker, and several production-grade requirements remain to be implemented or verified. The next milestone is runtime validation, followed by targeted completion of the missing architecture and quality gates.

Already Implemented

Backend and API

- FastAPI service with CORS and health endpoint.
- JWT login for seeded users.
- Mock KYC verification with PAN/Aadhaar selection and consent recording.
- SQLAlchemy models for users, accounts, transactions, features, scores, recommendations, alerts, audits, KYC, chat messages, and policy documents.
- Mock ledger with balance updates for posted transactions.
- Required API routes for login, KYC, transactions, dashboard, alerts, offers, chat, explainability, admin simulation, and health.
- Alembic environment and initial schema migration.
- Docker Compose startup migration command.

Fraud, ML, and Ethics

- Merchant category classification.
- India-time night detection.

- Geo-jump, new-device, velocity, watchlist, and night high-value rules.
- IsolationForest anomaly scoring on synthetic normal transaction data.
- XGBoost life-stage classifier with a deterministic fallback when XGBoost is unavailable.
- Segments: FIRST_JOB, MARRIAGE, MEDICAL, STRESS, SAVER, HIGH_VELOCITY, BASELINE.
- Stress detection and cash-flow-first behavior.
- Ethics gate that blocks active personal-loan recommendations for stressed users.
- SIP, micro-insurance, EMI conversion, starter credit, and grace-period recommendations.

Chat and RAG

- Persistent user and assistant chat messages.
- Policy corpus stored in backend/app/products.md.
- Lightweight keyword retrieval with trace output.
- Optional server-side Gemini integration.
- Safe fallback when the API key, HTTP client, or provider is unavailable.
- Stress-aware instruction that prevents credit upselling.

Frontend and Demo

- Next.js App Router PWA shell.
- Mobile-first banking dashboard.
- Login and seeded demo access.
- Consent-gated mock KYC screen.
- Balance, segment, savings, activity, offers, alerts, chat, and auditor views.
- Live transaction simulator with amount, location, device, and timestamp controls.
- Dockerfile, manifest, environment example, and responsive styling.

Quality and Operations

- Focused fraud and anti-predatory rule tests.
- Python compilation checks completed.
- GitHub commits and pushes completed through commit c9bd102.
- Secrets, databases, caches, and build output excluded by ignore files.

Required Before Calling the Project Complete

Priority 0: Runtime Setup and Smoke Test

This is required before more code changes have value.

1. Install Node.js 20 LTS and npm.
2. Install Docker Desktop and start the Docker engine.
3. Pull the latest branch:
4. Start the complete stack:

5. Verify:

- http://localhost:3000 loads.
- http://localhost:8000/health returns {"status":"ok","db":"ok"}.
- http://localhost:8000/docs opens.
- Login, KYC, dashboard, chat, explainability, and simulator work.

Priority 1: Fix Runtime Failures

Resolve actual failures from the first run, especially:

- PostgreSQL connection timing.
- Alembic migration behavior against pgvector.
- XGBoost installation and startup time.
- API-to-frontend CORS behavior.
- Mobile layout and browser console errors.
- Balance updates and blocked transaction behavior.
- Seed idempotency after repeated container starts.

Priority 2: Complete the Spec-Level Data and ML Contracts

1. Add the embedding VECTOR(768) column to the documents table.
2. Enable the vector PostgreSQL extension in the migration.
3. Add pgvector cosine indexes.
4. Add a real embedding provider adapter using Gemini embeddings or an equivalent provider.
5. Store document embeddings during corpus ingestion.
6. Replace keyword retrieval with vector retrieval while retaining a fallback.
7. Persist trained IsolationForest and XGBoost artifacts under backend/app/ml/models/.
8. Add a reproducible offline training script that generates at least 2,000 users and 80,000 synthetic transactions.
9. Add explicit model version metadata to audit records.

Priority 3: Complete Chat Tools and Safety Controls

Implement server-side tools with tests:

- get_balance
- get_offers
- get_last_txns
- request_restructure

Required behavior:

- The model must never write directly to the database.
- Restructure creates GRACE_PERIOD only for stressed users.
- Credit refusal is logged in audit_logs.
- Retrieved policy sources are returned in the trace.
- Aadhaar/PAN numbers must never be sent to the LLM provider.
- Provider timeouts and malformed responses must always fall back safely.

Priority 4: API and Security Hardening

- Add consistent {error, code, details} responses for every failure path.
- Add request validation for maximum transaction amounts and valid coordinates.
- Prevent debit transactions from reducing balances below the demo policy limit.
- Add authentication rate limiting for login.
- Add chat and simulator rate limits.
- Move secrets fully to environment configuration.
- Disable unsafe default JWT secrets outside local development.
- Add structured logging without sensitive document data.
- Add explicit authorization rules for admin simulation.
- Add transaction ownership checks to every detail endpoint.

Priority 5: Test Coverage

Add and run:

- API tests for login, invalid login, health, KYC consent, transaction posting, and blocked transactions.
- Seed idempotency test.
- Dashboard JSON serialization test.
- Balance update and blocked-balance invariants.
- Fraud scenario test using ■48,000, night time, geo jump, and new device.
- Stress user never receiving an active personal-loan offer.
- Explainability payload tests for blocked transaction and stressed user.
- Chat persistence and fallback tests.
- Gemini timeout/error fallback test.
- Alembic upgrade test on a clean database.
- Frontend production build.
- Browser smoke tests for login, KYC, dashboard, chat, and simulator.

Priority 6: Submission and Demo Polish

- Add a judge walkthrough document with a five-minute script.
- Add screenshots or a short demo video.
- Add a visible audit table for judges.
- Add transaction detail navigation from the activity list.
- Add language switching behavior rather than display-only language labels.
- Add browser speech-to-text using Web Speech API if time allows.
- Add PWA install metadata and icons.
- Add loading, empty, and error states for every main view.
- Add accessibility labels and keyboard navigation.
- Confirm the app works at phone and desktop widths.
- Confirm no secrets, real customer data, or generated database files are committed.

What the User Needs to Run

Full Docker Run

Install Docker Desktop, start it, then from the repository root:

Open:

- Frontend: <http://localhost:3000>
- API docs: <http://localhost:8000/docs>
- Health: <http://localhost:8000/health>

Demo login:

- Phone: 9000000001
- PIN: 1234

Local Development Run

Backend:

Frontend, in a second terminal:

Open <http://localhost:3000>.

Optional Gemini Setup

Create backend/.env locally, never in chat or Git:

The application still works with fallback chat if this key is absent.

Definition of Done

The project is complete when all of the following are true:

- Docker Compose starts database, API, and frontend without manual fixes.
- Alembic creates the complete schema on a clean database.
- /health returns database health.
- All required routes return the documented contract.
- All five demo stories are reproducible.
- Fraud blocks the high-risk demo before balance debit.
- Stressed users receive grace support and no active personal-loan offer.
- Chat is policy-grounded, persisted, and safe without or with Gemini.

- The frontend production build passes.
- Backend and browser tests pass.
- No secrets or real identity data are committed.
- A judge can complete the demo using only the README instructions.

Current Decision Point

The codebase is at a good checkpoint for runtime validation. The next user action is to install Node.js 20 and Docker Desktop, run `docker compose up --build`, and report the first error or successful screen. Further implementation should be driven by those runtime results before adding more features.